

Zerocoin Studios is building cutting-edge video enhancement software where both local effects (like blurring) and global tonal adjustments are critical. Mastering efficient, high-performance execution of these operations on diverse GPU hardware via OpenCL is fundamental to our product.

Task (2 Days Coding, 0.5 Day Report):

Your goal is to implement a command-line tool (or a minimal project) that takes an input image, processes it through two distinct stages on the GPU using OpenCL, and saves the enhanced image. You are encouraged to utilize a language you are comfortable with (C++ or Python) for the host-side OpenCL code.

1. Environment Setup & OpenCL Fundamentals (Self-driven, supported by boilerplate):

- Set up your development environment for OpenCL (e.g., C++ with appropriate headers/libraries, or Python with `pyopencl`).
- Familiarize yourself with the basic OpenCL model: Platforms, Devices, Contexts, Command Queues, Buffers, Programs, Kernels, Work-groups, Global IDs/Local IDs, and **OpenCL C for writing kernels**.
- *Leverage any boilerplate code* to get started quickly with OpenCL device/context setup, focusing your efforts on buffer management and kernel implementation.

2. Image Data Handling:

- Load a `.png` or `.bmp` image (e.g., 1920x1080) from disk into a CPU-side `UInt8` array (representing RGBA pixel data). You may use a third-party library like `stb_image.h` (C/C++) or Python's PIL/Pillow for simplified image I/O.
- Efficiently transfer this raw image data from CPU memory to an `cl_mem` buffer on the GPU.
- After GPU processing, transfer the final processed image data back from the GPU to CPU memory.
- Save the final processed image (from the CPU-side buffer) back to a `.png` or `.bmp` file.

3. OpenCL Compute Kernel Development (OpenCL C):

- Write **two separate OpenCL C compute kernels** within a single `.cl` file. These kernels will operate sequentially.

▪ Kernel 1: Per-Channel 3x3 Gaussian Blur

- Input: An RGBA 8-bit per channel image buffer.
- Output: A blurred RGBA 8-bit per channel image buffer.
- Apply a standard 3x3 Gaussian blur filter separately to each of the R, G, and B channels. The Alpha channel should remain untouched.
- Use the following 3x3 Gaussian kernel (normalized):

1	2	1
2	4	2
1	2	1
- --- ---
- Total Sum = 16
-

(Each element in the kernel should be divided by 16.0f).

- **Crucially:** Implement robust **boundary handling** (e.g., clamping coordinates to image bounds or replicating border pixels) to avoid out-of-bounds memory access for pixels near the image edges.

- **Kernel 2: Logarithmic Tone Mapping**

- Input: The blurred RGBA image produced by Kernel 1. (Assume this image is still in a 0-255 integer range but you'll work with normalized float values [0,1] for calculations).
- Output: An 8-bit per channel image with logarithmic tone mapping applied to its luminance.
- **Steps:**
 1. Convert the input RGBA pixel (R, G, B values, normalized to [0,1]) to a single luminance value (Y) using the standard luminosity formula:
$$Y = 0.2126 \times R + 0.7152 \times G + 0.0722 \times B$$
 2. Apply the logarithmic tone mapping formula to this luminance:
$$Y_textout = \frac{\log(1.0f + Y)}{\log(1.0f + \text{max_luminance})}$$
 - **max_luminance** should be a `float` parameter passed from the host (e.g., `1.0f`, `5.0f`, or `10.0f` to control compression).
 3. **Crucially:** Reapply this `Y_textout` back to the R, G, and B channels while preserving their original color ratios. A common way is `R_textout = R * (Y_textout / Y)`, `G_textout = G * (Y_textout / Y)`, etc. (handle `Y=0` case to avoid division by zero).
 4. Ensure final RGB pixel values are clamped to the valid 8-bit range [0, 255] and the Alpha channel is preserved.
- **Note:** This requires careful float arithmetic and conversion back to `UInt8`.

4. OpenCL Host Code Execution:

- Implement the host-side (CPU) OpenCL code to:
 - Utilize the boilerplate code to initialize OpenCL.
 - Compile your OpenCL kernels from the `.cl` file.
 - Create input, output, and an intermediate `cl_mem` buffer to pass data between kernels.
 - Set kernel arguments for both kernels, including the **max_luminance** parameter for Kernel 2.
 - Enqueue Kernel 1, then enqueue Kernel 2, ensuring proper synchronization (e.g., using `cl_event` objects or `clFinish()`) so that Kernel 2 starts only after Kernel 1 completes.
 - Read the final processed image data back from the GPU.

Deliverables (By the end of 2.5 days):

1. **Code Repository:** A link to a public Git repository (GitHub/GitLab/Bitbucket) containing:

- All source code (C++ or Python host code, `.cl` OpenCL kernel file).
- A clear README `.md` with detailed instructions on how to compile and **run** the project on a common operating system (Linux, macOS, or Windows) with OpenCL drivers. List any required libraries or tools (e.g., `stb_image.h`).
- The input image provided.
- The generated output image(s) from your program.
- **Crucially:** Commits showing progressive development, reflecting your thought process and efforts to bring the project to closure.

2. **One-Page Report (PDF):** A concise, single-page report covering:

- **Technical Approach & Kernel Design:**
 - Describe your overall OpenCL pipeline.
 - Explain your approach to implementing the Gaussian blur, specifically detailing your **boundary handling strategy** and why you chose it.
 - Briefly explain the Logarithmic Tone Mapping formula and how you re-applied the adjusted luminance to maintain color ratios.
- **Challenges, Learning, & Problem-Solving:**
 - Describe the most significant technical challenges you encountered (e.g., OpenCL setup, debugging kernels, handling data types, synchronization).
 - Explain *how you systematically debugged and overcame* these challenges.
 - What new concepts or skills did you learn about GPU programming (specifically OpenCL) during this assignment?
- **Parallelism & Performance Considerations:**
 - Explain how the Gaussian blur and tone mapping operations are parallelized across the GPU.
 - Theoretically discuss factors (e.g., memory access patterns for blur, computational intensity, work-group size choice) that might influence performance for these specific operations. (No actual benchmarking required).
- **Project Closure & Future Work:**
 - Reflect on your ability to bring this project to a working state within the given timeframe. What strategies did you employ for time management?
 - Suggest one or two key ways this project's core functionality could be significantly extended or optimized for a real-world video enhancement product using OpenCL (e.g., larger kernel sizes, different filters, advanced color models, image objects, shared memory optimization).